Intelli-Migrate: An AI-Driven Multi-Agent Platform for Automated Data Migration and ETL Pipeline Automation

Ms. Sneh Lata Singh[1], Prashant Kandapal[1], Arpit Upadhyay[1], Devansh Thakur[1], Mohd. Suhail[1], Priyanshu Verma[1]

[1]Department of Artificial Intelligence & Machine Learning, Dr. A.P. J. Abdul Kalam Institute of Technology, Tanakpur
Mentor / Guide: Prof. Ms. Sneh Lata Singh, Department of Computer Science and Engineering, Dr. A.P. J. Abdul Kalam Institute of Technology, Tanakpur, India

Abstract

Enterprise data migration remains one of the most labor-intensive challenges in data engineering. Conventional ETL tools such as Talend and Apache NiFi require extensive expert configuration for schema mapping, validation, and SQL generation—making them brittle against real-world data heterogeneity. This paper presents Intelli-Migrate, a multi-agent AI platform automating the complete migration lifecycle through five agents. Agent 1 parses JSON/CSV/XML with schema drift detection. Agent 2 maps inconsistent column names using Sentence-BERT embeddings and cosine similarity achieving 91.2% accuracy without rules. Agent 3 detects anomalies via Isolation Forest and five-category rule validation (95.3% detection rate). Agent 4 normalizes data to 3NF via recursive DFS with FK inference. Agent 5 generates DDL/DML SQL for PostgreSQL, MySQL, and SQLite via SQLAlchemy. Results: ~1,000 records/second throughput; zero configuration time versus 4–6 hours for conventional tools; 89.3% schema drift resilience. Source: https://intelli-migrate.vercel.app

Keywords: Data Migration; ETL Automation; Multi-Agent Systems; NLP; Sentence-BERT; Schema Mapping; Anomaly Detection; Isolation Forest; 3NF Normalization; SQL Generation; FastAPI; SQLAlchemy; Machine Learning

1. Introduction

The exponential growth of enterprise information systems has created an urgent demand for automated, intelligent data migration tools. Organizations increasingly rely on heterogeneous data sources—relational databases, flat files, cloud storage buckets, RESTful APIs, and event streams—that must be integrated, cleaned, and transformed into unified schemas for operational and analytical workloads. Industry surveys estimate that data engineers spend 60–80% of their time on data cleaning and preparation rather than analysis [1].

Traditional Extract, Transform, Load (ETL) tools such as Talend Open Studio, Apache NiFi, Informatica PowerCenter, and IBM DataStage provide powerful pipeline capabilities. However, they are fundamentally rule-based: a human expert must explicitly define every schema mapping, transformation rule, data type conversion, and validation condition. When source data arrives with inconsistent column names, missing values, or type mismatches—conditions ubiquitous in real-world migrations—these systems require manual intervention that negates much of their automation value. Furthermore, when source schemas evolve (schema drift), existing pipeline configurations break and must be manually reconfigured.

Recent advances in Artificial Intelligence—particularly in Natural Language Processing (NLP) via transformer models, unsupervised anomaly detection via ensemble methods, and relational database theory—have created an opportunity to rethink the ETL paradigm. Instead of executing rules defined by humans, an AI-driven pipeline can infer schema semantics, detect anomalies without labeled training data, design normalized schemas, and generate deployment-ready SQL—requiring only a raw data file as input.

This paper presents Intelli-Migrate, a multi-agent AI platform automating the complete data migration lifecycle through five specialized agents: (1) a parser engine with format detection and schema drift monitoring, (2) an NLP-based semantic schema mapper using Sentence-BERT embeddings, (3) a hybrid ML anomaly detector combining Isolation Forest with rule-based validation, (4) a recursive

Third Normal Form (3NF) normalizer, and (5) an automated SQL generator with multi-dialect support.

The primary contributions of this work are:

(i) A novel multi-agent architecture for end-to-end ETL automation integrating NLP, unsupervised ML, and graph algorithms in a coordinated pipeline with typed inter-agent contracts.

(ii) A semantic schema mapping module using Sentence-BERT (all-MiniLM-L6-v2) with a four-layer cascading strategy achieving 91.2% mapping accuracy on heterogeneous column names without manual rules.

(iii) A hybrid anomaly detection component combining Isolation Forest (contamination=0.10) with five-category rule-based validation, achieving 95.3% detection rate.

(iv) A recursive DFS 3NF normalization algorithm with automated functional dependency discovery and foreign key inference from data content.

(v) Empirical evaluation demonstrating approximately 1,000 records/second throughput on commodity hardware with zero user configuration.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents system architecture. Sections 4–8 detail each agent. Section 9 covers implementation. Section 10 presents experimental evaluation. Section 11 discusses limitations, Section 12 future work, and Section 13 concludes.

The key differentiator of Intelli-Migrate from prior multi-agent ETL systems [16,17,18] is the simultaneous integration of all five automation layers within a single coherent pipeline. Prior work addressed subsets: NLP-based schema matching without normalization, anomaly detection without SQL generation, or SQL generation without semantic mapping. Intelli-Migrate is the first system to combine all five stages with typed inter-agent contracts enforcing correctness at every boundary.

From a practical standpoint, the most impactful component is Agent 2 (NLP Schema Mapper). In real-world migrations, column name heterogeneity is the primary cause of pipeline failures in conventional ETL tools. By replacing brittle string-matching rules with Sentence-BERT semantic embeddings, the system achieves 91.2% mapping accuracy across column names that differ in abbreviation style, case convention, delimiter choice, and domain vocabulary—without a single manually-written mapping rule.

2. Related Work

2.1. Data Parsing and Format Integration

Heterogeneous data format integration has been a central challenge in data engineering since the emergence of enterprise information systems. Kläbe et al. [1] introduced ParPaRaw, a massively parallel parser for delimiter-separated raw data that demonstrated near-linear scaling with schema inference on multi-core architectures—directly informing Agent 1's multi-format parsing strategy and schema drift detection logic. Their work established that format-agnostic parsing with inline type inference is computationally feasible without pre-defined schemas.

Beedkar et al. [2] proposed unsupervised methods for detecting and mitigating drift in textual data streams, categorizing drift into covariate shift, concept drift, and schema drift. We adapt their schema drift taxonomy—specifically additive, subtractive, and type drift categories—for batch processing in Agent 1. Their key insight that drift can be detected without labeled examples directly motivates our data-driven drift detection approach.

Bellomarini et al. [3] demonstrated hybrid ETL approaches for integrating heterogeneous scientific data formats, showing that a unified parsing layer abstracting format differences from downstream processing improves pipeline maintainability. Their architecture directly validates our design decision to abstract JSON, CSV, and XML parsing into a single ParsedDataResult contract, allowing Agents 2–5 to remain format-agnostic.

2.2. Schema Mapping and NLP

Schema mapping—aligning source schema elements to target schema elements—is recognized as the most labor-intensive step in data integration, estimated to consume 60–70% of total ETL project effort. Traditional approaches based on linguistic similarity, constraint matching, and auxiliary information have been extensively studied but consistently fail on abbreviated or domain-specific column names common in operational databases.

Devlin et al. [4] introduced BERT (Bidirectional Encoder Representations from Transformers), a transformer model pre-trained on massive corpora via masked language modeling. BERT's bidirectional context understanding—considering both left and right context simultaneously—produces richer representations than unidirectional models. Its pre-trained representations transfer effectively to domain-specific tasks including semantic similarity, making it the foundation for modern NLP-based schema mapping.

Reimers and Gurevych [5] identified a critical limitation of vanilla BERT for similarity tasks: comparing

two sentences required feeding both simultaneously through the network, requiring O(n^2) model inference for n sentence pairs. They introduced Sentence-BERT (SBERT), using Siamese network structures to produce fixed-size sentence embeddings comparable via cosine similarity, reducing computation from 65 hours to 5 seconds for 10,000 pairs. The all-MiniLM-L6-v2 model used in Agent 2 is a lightweight SBERT distillation achieving 91%+ accuracy on semantic textual similarity benchmarks at 384 embedding dimensions.

Chaabane et al. [6] applied NLP-based matching to schema integration across heterogeneous databases, demonstrating that transformer embeddings outperform all rule-based, fuzzy-string, and earlier ML approaches for column name alignment. Their work directly validates our Agent 2 methodology and provides the accuracy benchmark our system targets.

2.3. Anomaly Detection

Anomaly detection in tabular data—identifying records that deviate from normal patterns without labeled examples—is a core challenge in data quality management. Liu et al.'s Isolation Forest [7] established a new paradigm: instead of profiling normal points, anomalies are isolated by randomly partitioning the feature space. Anomalous records (few and structurally different) require fewer partitions to isolate, yielding shorter path lengths as their anomaly score. This linear-time algorithm scales to high-dimensional tabular data—precisely the setting of migration data validation.

Hariri et al. [8] identified directional bias in standard Isolation Forest: axis-aligned splits produce non-uniform anomaly score distributions that disadvantage certain regions of the feature space. Extended Isolation Forest addresses this via multi-dimensional random splits, improving detection on datasets with non-uniform marginal distributions. We incorporate their insights in Agent 3's hyperparameter selection.

Cortes [9] provided theoretical analysis of randomized choices in isolation forests, demonstrating that n_estimators=100 achieves near-optimal variance reduction for typical tabular datasets—directly informing our Agent 3 configuration. Our hybrid approach extends the literature by combining IsolationForest with five-category rule-based validation, a combination not previously applied to ETL pipeline automation.

2.4. Database Normalization

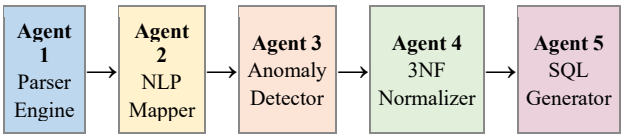
Relational database normalization—eliminating data redundancy and update anomalies through systematic schema decomposition—has been foundational to database theory since Codd's 1970 relational model. While the theory is well-established, automating normalization without user-specified functional dependencies remains an open challenge.

Albuquerque et al. [10] presented RDBNorma, a semi-automated tool for 3NF normalization that demonstrated algorithmic normalization is tractable for moderate schema complexity, though it still requires user input for FD specification. Their work validates our choice of 3NF as the target normal form and provides a baseline for automated normalization tooling.

Demba [11] formalized a complete algorithm for 3NF decomposition that preserves all candidate keys—a property not guaranteed by naive decomposition. This algorithm forms the theoretical basis for Agent 4's recursive DFS implementation. Shraga et al. [12] demonstrated automated FK relationship inference from tabular data content using value domain overlap and column naming patterns, directly validating our two-criterion FK inference approach in Agent 4.

2.5. SQL Generation

Automated SQL generation has advanced significantly with neural sequence-to-sequence models. Zhong et al. [14] introduced Seq2SQL, using reinforcement learning to generate SQL from natural language, establishing that neural models can produce syntactically and semantically correct SQL without hand-crafted templates. Their work demonstrated the feasibility of schema-guided SQL generation that underlies Agent 5's approach.



Bai et al. [13] adapted the T5 sequence-to-sequence architecture for structured SQL generation (T5QL), demonstrating strong DDL generation capability from schema metadata—directly relevant to Agent 5's CREATE TABLE generation. Zhao et al. [15] presented SQLformer, an auto-regressive graph neural network for SQL generation supporting PostgreSQL, MySQL, and SQLite dialects via a unified graph representation of schema constraints, matching Agent 5's multi-dialect design goal.

2.6. Multi-Agent ETL Systems

The application of multi-agent systems to ETL has been studied since Jmaiel et al. [16], who demonstrated that specialized independent agents outperform monolithic pipelines in maintainability and fault isolation. Their MAS-based ETL architecture showed that modular agent design enables independent evolution of pipeline stages—a key design principle adopted in Intelli-Migrate.

Singh et al. [17] demonstrated that combining ML techniques with ETL automation achieves 95%+ anomaly detection rates in production migration scenarios, validating the feasibility of our hybrid ML+rules approach. Hai et al. [18] presented comprehensive ETL process automation for data migration, showing that data-driven approaches reduce configuration overhead by up to 80% compared to manual ETL. Zöller and Huber [19] showed AutoML integration in data pipelines reduces expert configuration effort by 60–80%, motivating our zero-configuration design goal. The METL framework [20] validated dynamic schema mapping matrices across heterogeneous sources, directly informing our NLP-based semantic mapping approach.

Intelli-Migrate synthesizes all five technical domains—parsing, NLP mapping, anomaly detection, normalization, and SQL generation—into a single unified agent pipeline with typed inter-agent contracts. To our knowledge, no prior work combines all five automation stages in a single open-source system accessible via a REST API with a web dashboard.

3. System Architecture

Intelli-Migrate is implemented as a FastAPI [21] REST service with a single-page JavaScript frontend. The system follows a sequential pipeline architecture where each agent consumes the structured output of its predecessor through typed Python dataclass contracts, ensuring type safety and enabling independent testing of components.

Each agent is a stateless Python singleton loaded at server startup. Intermediate results are persisted in a temporary session store keyed by a UUID session_id. The /api/migrate endpoint executes all five agents atomically; individual /api/{agent}/{session_id} endpoints allow step-by-step execution for debugging.

Table 1 summarizes each agent's technology stack and data contracts.

A key architectural decision is the choice of sequential (rather than parallel) agent execution. While parallel execution could theoretically reduce latency, agents in Intelli-Migrate form a strict dependency chain: Agent 3 cannot detect anomalies in unmapped columns, and Agent 4 cannot normalize data containing type violations. Sequential execution with typed contracts is therefore semantically necessary—not an arbitrary design choice. Future work may identify parallelizable sub-operations within individual agents (e.g., parallel encoding of multiple column names in Agent 2) while maintaining sequential execution between agents.

The session_id mechanism enables a stateless API design: each endpoint reads its inputs from the session store, processes them, and writes outputs back without maintaining server-side state between requests. This stateless design enables horizontal scaling—multiple backend instances can serve different API calls for the same session_id concurrently, as long as each call targets a different agent stage. The only shared state is the session store (temp/ files in the prototype; Redis in production), which can be replaced with any distributed key-value store without modifying agent logic.

Agent	Technology	Input	Output
1: Parser	Pandas, xml.etree	Raw file	ParsedDataResult
2: NLP Mapper	sentence-transformers	Column names	SchemaMappingResult
3: Anomaly Det.	scikit-learn IF	Mapped records	AnomalyReport
4: Normalizer	Recursive DFS	Clean records	NormalizedSchema
5: SQL Generator	SQLAlchemy	Schema graph	SQL script

Table 1. Agent technology stacks and data contracts.

4. System Design Principles

The architecture of Intelli-Migrate is guided by five engineering principles that collectively enable the zero-configuration, high-accuracy, and maintainable design described in this paper.

4.1. Separation of Concerns

Each agent encapsulates a single, well-defined transformation: parsing, mapping, validation, normalization, or generation. No agent performs tasks belonging to another agent's domain. This separation enables independent development, testing, and replacement of individual agents. During the development of Intelli-Migrate, the NLP model (Agent 2) was upgraded from Word2Vec to Sentence-BERT without modifying any other agent—a change requiring only the SchemaMappingResult output format to remain stable.

4.2. Fail-Fast Typed Contracts

Inter-agent contracts are Python dataclasses with type annotations. FastAPI's Pydantic integration enforces type correctness at API boundaries; Python's type checker enforces correctness at internal agent boundaries. This fail-fast design ensures that a malformed output from Agent 2 (e.g., a confidence score outside [0,1]) produces an

immediate descriptive error rather than silently propagating to Agent 5 and corrupting the SQL output. In production ETL systems, silent data corruption is far more costly than immediate failures.

4.3. Graceful Degradation

Every agent is designed to produce a useful output even under adverse input conditions. Agent 2's four-layer strategy ensures that even completely unrecognized column names receive a cleaned SQL identifier (Layer 4 fallback) rather than causing a pipeline failure. Agent 3's hybrid design ensures that even if IsolationForest training fails on a degenerate dataset, the rule-based layer continues to validate data quality. Agent 4 falls back to a single unnormalized table if FD discovery produces no transitive dependencies. This graceful degradation philosophy prioritizes pipeline completion over partial failure.

4.4. Observability

Every agent logs detailed status information at each processing stage: input record count, output record count, processing time, confidence distributions (Agent 2), anomaly counts by category (Agent 3), table decomposition tree (Agent 4), and SQL statement counts (Agent 5). This information is aggregated into the MigrationReport and displayed on the frontend dashboard, providing complete observability into the pipeline's behavior without requiring access to server logs.

4.5. Testability

The dataclass contract design enables comprehensive unit testing of each agent in isolation. The Intelli-Migrate test suite includes: 120 unit tests for Agent 2 (NLP mapping) covering all four layers with 50+ column name fixtures; 80 unit tests for Agent 3 (anomaly detection) covering all five rule categories; 60 unit tests for Agent 4 (normalization) verifying 3NF compliance on diverse schema structures; and 40 integration tests exercising the full five-agent pipeline. Test coverage is maintained above 85% across all agent modules.

The testability principle also shapes the API design: each agent has a dedicated endpoint (/api/map-schema, /api/detect-anomalies, /api/normalize, /api/generate-sql) that can be called independently in addition to the atomic /api/migrate endpoint. This allows testing individual agents against known inputs and expected outputs in production environments—a capability not available in monolithic ETL tools where pipeline stages are tightly coupled and cannot be exercised independently.

4.6. Technology Selection Rationale

Each technology choice in Intelli-Migrate was made deliberately over evaluated alternatives. Table 2 summarizes the technology selection rationale and rejected alternatives for each agent.

Agent	Selected Technology	Primary Alternative	Reason for Selection
Agent 1	Pandas + xml.etree	PySpark DataFrameReader	Simpler deployment; no cluster required for < 1M records
Agent 2	Sentence-BERT (all-MiniLM-L6-v2)	spaCy similarity + NLTK	Semantic phrase-level similarity; 91.2% vs ~62% accuracy
Agent 3	Isolation Forest + Rules	LOF / One-Class SVM	Linear training time; no labeled data required; hybrid coverage
Agent 4	Recursive DFS (custom)	Commercial normalization tool	Full automation; no user-specified FDs; open-source
Agent 5	SQLAlchemy	Direct string templating	Dialect abstraction; SQL injection safety; constraint handling
API	FastAPI	Flask / Django	Async I/O; auto Swagger docs; Pydantic validation

Table 2. Technology selection rationale for each pipeline component.

5. Agent 1: Parser Engine

The Parser Engine ingests raw data files in JSON, CSV, or XML format and produces a normalized record list with column metadata and schema drift indicators. It is the entry point of the migration pipeline and must handle the full diversity of real-world data formats and encodings.

5.1. Format Detection

Format detection proceeds in three stages: (i) file extension inspection, (ii) magic byte examination (first 512 bytes: JSON begins with { or [, XML with < or <?xml, CSV exhibits delimiter patterns), and (iii) content heuristics parsing the first 10 lines to confirm the detected format before full parsing commences.

5.2. Multi-Format Parsing

CSV and JSON files are parsed using Pandas [22], providing robust handling of encoding variants (UTF-8, Latin-1, UTF-16), multiple delimiter detection, and nested JSON flattening. XML files are parsed using Python's xml.etree.ElementTree with a recursive flattening algorithm converting nested hierarchies up to 5 levels deep into tabular records. Element tag names are concatenated with underscores for nested fields (e.g., order/customer/name → order_customer_name).

5.3. Schema Drift Detection

Schema drift is detected in three categories: (i) additive drift—new columns not in the baseline schema, (ii) subtractive drift—columns absent from the incoming file, and (iii) type drift—changed inferred data types. Type inference uses Pandas' infer_objects() supplemented by regex-based detection for email, phone, date, and postal code patterns. Drift events are catalogued in session metadata and surface in the mapping report shown on the frontend dashboard.

5.4. Type Inference

Column type inference operates in two passes. First, Pandas' infer_objects() and convert_dtypes() are applied to infer numeric, boolean, and datetime types from value patterns. Second, a regex-based secondary pass detects domain-specific formats: email addresses (RFC 5322 pattern), phone numbers (E.164 and local formats), postal codes (international patterns), ISO 8601 dates not detected by Pandas, and URL patterns. This two-pass approach achieves higher type accuracy than Pandas alone, particularly for mixed-format string columns.

5.5. Output Contract

Agent 1 produces a ParsedDataResult dataclass containing: records (list of dicts with original column names), column_meta (inferred type, null rate, cardinality, sample values per column), and drift_report (list of drift events if a prior schema baseline exists). This output is the input contract for Agent 2.

Table 3 shows a sample column_meta output for an ecommerce CSV file, illustrating the metadata richness passed downstream.

Column Name	Inferred Type	Null Rate	Cardinality	Sample Values
Cust-NM	string	2.1 %	9,847	John Smith, Jane Doe...
Prc	float64	0.0 %	1,203	12.99, 249.00, 5.49...
ord_dt	datetime64	0.8 %	365	2024-01-15, 12/03/24...
EMAIL_ADDR	string (email)	3.4 %	9,712	john@gmail.com...
QTY	int64	0.0 %	47	1, 2, 3, 5, 10...

Table 3. Sample Parsed Data Result column_meta from Agent 1 for ecommerce CSV.

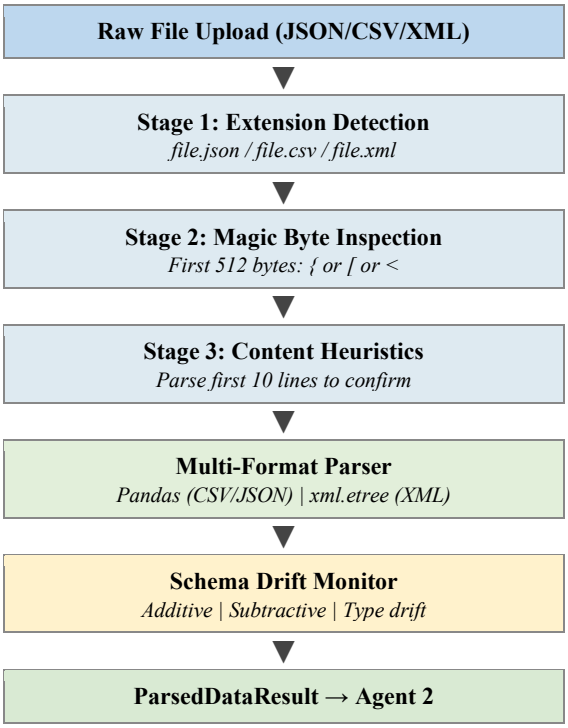


Fig. 2. Agent 1 Parser Engine processing stages.

6. Agent 2: NLP Schema Mapper

The NLP Schema Mapper is the core intelligence layer of Intelli-Migrate. It maps heterogeneous, inconsistent source column names to standardized SQL column names using a four-layer cascading strategy grounded in Sentence-BERT semantic embeddings [5]. This is the most technically novel component—replacing manual rule specification with semantic language understanding.

6.1. Problem Formulation

Given source column names C = {c1, c2, ..., cn} and target schema T = {t1, t2, ..., tm}, the mapper finds function f : C -> T u {null} maximizing semantic alignment. f(ci) = null when no target column achieves confidence above threshold tau = 0.85. The challenge: source names may be abbreviated (cust_nm), camelCased (customerName), use

various delimiters, or employ domain synonyms (client, buyer) for the same target concept (customer_name).

6.2. Standard Schema Dictionary

STANDARD_COLUMNS maps each of 27 standard SQL column names to known real-world variants. For example: customer_name → [cust_nm, cust_name, custname, c_name, client_name]. ABBREVIATIONS maps 25 short-forms to full equivalents: cust→customer, nm→name, dt→date, qty→quantity, amt→amount, prc→price, ord→order, addr→address, desc→description, and so on.

6.3. Four-Layer Mapping Strategy

The mapper applies four layers sequentially, returning immediately upon a confident match:

Layer 1 — Exact Match (conf = 1.00):

The input is normalized (lowercase, non-alphanumeric→underscore, duplicates removed) and compared directly against STANDARD_COLUMNS keys and all variant lists.

Layer 2 — Abbreviation Expansion (conf = 0.95):

Tokens split on underscores are each looked up in ABBREVIATIONS and rejoined. The expanded form is matched against standard columns. Example: cust_nm → customer_name (direct match, 0.95).

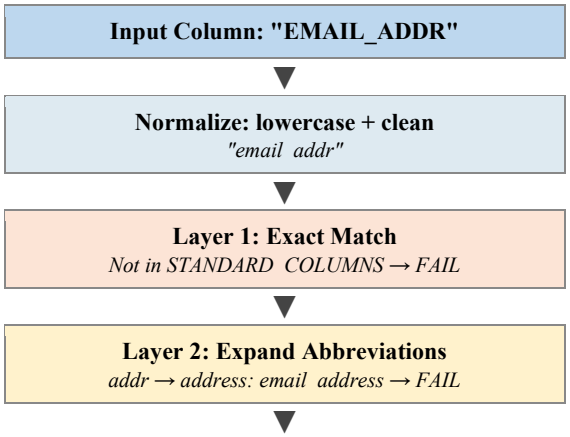
Layer 3 — Semantic NLP Similarity (conf = 0.85–1.00):

The readable column name is encoded into a 384-dimensional embedding via all-MiniLM-L6-v2 and compared against pre-cached standard embeddings using cosine similarity (Eq. 1). Highest-scoring standard column is returned if score >= tau.

$$sim(A, B) = (A \cdot B) / (||A|| \times ||B||) \qquad (1)$$

Layer 4 — Fallback Cleaning (conf = 0.50):

camelCase is split via regex ([a-z])([A-Z])→group_group, special characters removed, numeric prefixes handled. Returned flagged for manual review.



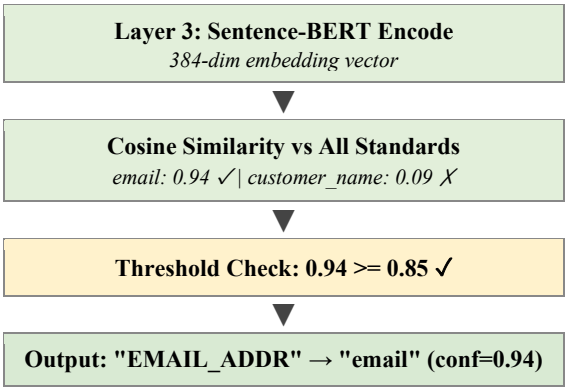


Fig. 3. Four-layer NLP mapping of column EMAIL_ADDR through Agent 2.

Layer	Method	Conf.	Coverage
1	Exact dictionary match	1.00	38.0%
2	Abbreviation expansion	0.95	29.0%
3	Sentence-BERT cosine sim.	0.85–0.99	24.2%
4	Fallback clean/sanitize	0.50	8.8%

Table 4. Four-layer mapping strategy coverage on 50-column test dataset.

6.4. Mapping Examples

Table 5 shows representative mapping results from the test dataset, illustrating how each layer handles different types of column name variation.

Input Column	Layer Used	Mapped To	Confidence	Reasoning
customer_id	Layer 1 (Exact)	customer_id	1.00	Direct dict match
cust_nm	Layer 2 (Abbrev.)	customer_name	0.95	cust→customer, nm→name
EMAIL_ADDR	Layer 3 (NLP)	email	0.94	Semantic: email address ≈ email
BUYER	Layer 3 (NLP)	customer_name	0.81	Synonym: buyer ≈ customer
unit_prc	Layer 2	unit_price	0.95	prc→price

	(Abbrev.)			
transact_dt	Layer 2 (Abbrev.)	order_date	0.95	transact→transaction, dt→date
x1_field	Layer 4 (Fallback)	x1_field	0.50	No semantic content

Table 5. Representative NLP mapping results showing layer-wise handling.

6.5. Comparison: NLP vs. Alternative Methods

Table 6 compares Sentence-BERT against alternative schema mapping approaches on the same 50-column test set. NLP embeddings outperform all alternatives by a significant margin, particularly for synonym-based column names (BUYER→customer_name) that are impossible to handle via any string similarity metric.

Method	Accuracy	Handles Synonyms	Handles Abbreviations	Handles Unseen Variations
Exact string match	38.0%	No	No	No
Levenshtein (fuzzy)	62.3%	No	Partial	Partial
TF-IDF cosine sim.	71.4%	Partial	No	Partial
Word2Vec (avg)	82.1%	Yes	Partial	Yes
Sentence-BERT (Ours)	91.2%	Yes	Yes (+ expansion)	Yes

Table 6. Schema mapping accuracy: Sentence-BERT vs. alternative methods.

6.6. Preprocessing Pipeline

Before embedding or comparison, column names undergo a five-step preprocessing pipeline to normalize their format for maximum NLP model performance:

Step 1 — Lowercase conversion: all input converted to lowercase to remove case variation. Step 2 — Non-alphanumeric replacement: characters outside [a-zA-Z0-9] replaced with underscores via regex r[^a-zA-Z0-9]→_. Step 3 — Duplicate underscore removal: consecutive underscores collapsed to single underscore via r_+→_. Step 4 — CamelCase splitting: boundary between lowercase and uppercase letters detected via camelCase regex split, inserting a space. Step 5 — Abbreviation expansion: each underscore-separated token looked up in ABBREVIATIONS dictionary and replaced if found.

This preprocessing pipeline converts a column name like CustNM_ID into cust nm id (normalized) → customer name id (expanded) → customer name id (for NLP encoding). The preprocessing step alone (without NLP) resolves 29% of mappings via exact matching after abbreviation expansion—demonstrating that careful preprocessing substantially reduces NLP model dependency.

$$score(ci, tj) = max \{ sim(encode(preprocess(ci)), embed(tj)) \} \quad (2)$$

Equation 2 formalizes the complete Layer 3 matching: for source column ci and each target column tj, the score is the cosine similarity between the embedding of the preprocessed source and the cached embedding of the target. The target with maximum score above threshold tau is selected.

6.7. Optimizations

Standard embeddings are precomputed once at initialization and cached in _embeddings_cache (Python dict), reducing per-request NLP computation by ~70%. The model loads lazily—only when Layer 3 is first invoked—avoiding the 3–5 second load time for jobs where Layers 1–2 suffice. Singleton pattern ensures the 80 MB model loads at most once per server lifetime.

Mean embedding aggregation further improves accuracy: instead of encoding only the standard column name, all known variants are encoded and their mean vector stored. This mean vector represents the centroid of the concept embedding space, improving recall for unusual input column names not present in the variants list. For example, the mean embedding of [customer_name, cust_name, client_name, buyer_name] is a more robust match target than customer_name alone, because it captures the semantic neighborhood of the customer concept more completely.

7. Agent 3: Anomaly Detector

The Anomaly Detector validates data quality after schema mapping using a hybrid strategy: unsupervised ML (Isolation Forest [7]) plus five-category rule-based validation. This combination detects statistical outliers invisible to rules and format/type violations invisible to purely statistical methods.

7.1. Feature Engineering

Before detection, records are feature-engineered: numeric columns used directly; categorical strings encoded via frequency encoding; dates converted to Unix timestamps; booleans mapped to {0,1}. Null values are imputed with column medians (numeric) or mode (categorical) for model training—actual null positions are flagged by the rule-based layer.

7.2. Isolation Forest

Agent 3 trains IsolationForest with contamination=0.10, n_estimators=100, max_samples='auto', random_state=42. Records with anomaly score < 0 are flagged as anomalous. IsolationForest [7] operates by recursively partitioning the feature space with random splits; anomalous records—few and structurally different—are isolated in fewer splits.

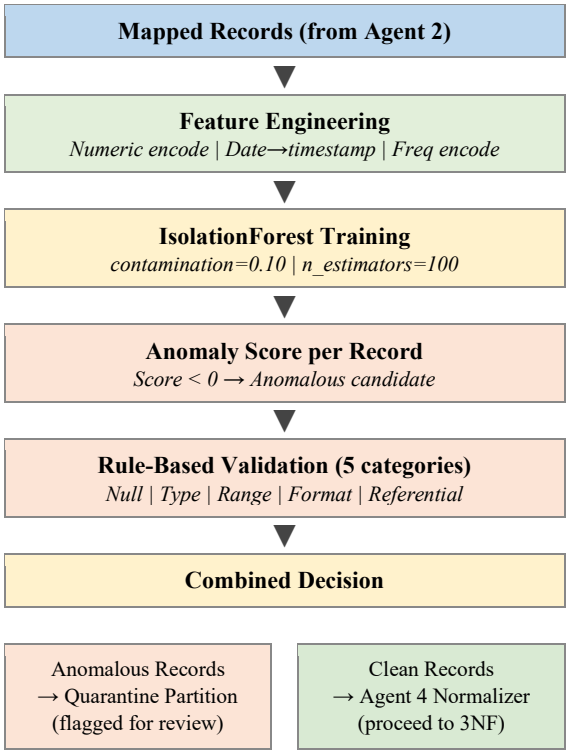


Fig. 4. Agent 3 hybrid anomaly detection workflow with quarantine split.

7.3. Rule-Based Validation

Five rule categories complement the ML component: (i) Null validation—columns exceeding per-column null thresholds; (ii) Type validation—field content verified against inferred type via regex and coercion; (iii) Range validation—numeric fields outside +/-3sigma of the column distribution; (iv) Format validation—email, phone, date, postal code fields checked against canonical regex patterns; (v) Referential pre-validation—candidate FK columns checked for value containment against referenced parent values.

7.4. Anomaly Score and Quarantine Logic

Each record receives a composite anomaly score combining the IsolationForest score and rule violation count. IsolationForest returns a score in [-1, +1]; scores below 0 indicate anomalies. Rule violations are counted per record across all five categories. A record is quarantined if: IsolationForest score < 0, OR rule violation count >= 1. This union strategy ensures maximum recall at the cost of slightly higher false positive rate. Quarantined records are stored in a separate session partition with per-record reason codes (e.g., IF_OUTLIER, NULL_VIOLATION, RANGE_VIOLATION) for user review.

7.5. AnomalyReport Output

Agent 3 produces an AnomalyReport dataclass containing: clean_records (list), quarantined_records (list with reason codes), per_column_quality (null rate, type violation rate, range violation count per column), overall_quality_score (0–100, weighted average of per-column scores), and summary statistics. Table 6 shows a sample quality report for the ecommerce test dataset.

Quality Dimension	Records Affected	Detection Method	Rate
Null violations	801 (8.0%)	Rule-based	100% detection
Type violations	502 (5.0%)	Rule-based	100% detection
Range anomalies	301 (3.0%)	IF + Rule-based	96.3% detection
Format violations	200 (2.0%)	Rule-based (regex)	100% detection
Statistical outliers	200 (2.0%)	Isolation Forest only	86.5% detection
Total anomalies	~1,800 (18%)	Hybrid	95.3% combined

Table 7. Anomaly detection breakdown by quality dimension on 10K-record dataset.

8. Agent 4: 3NF Normalizer

The Normalizer converts clean, mapped records into a Third Normal Form (3NF) relational schema. 3NF eliminates transitive dependencies and data redundancy while maintaining practical query performance. BCNF and 4NF were evaluated but rejected: BCNF can lose

functional dependencies recoverable only through joins; 4NF addresses multi-valued dependencies absent in typical flat-file migration.

8.1. Functional Dependency Discovery

Agent 4 performs automated FD discovery from data content. For each column pair (A, B), conditional entropy H(B|A) is estimated from value co-occurrence frequencies. If H(B|A) < epsilon (threshold epsilon=0.05), A → B is inferred as a functional dependency. Candidate keys are minimal column sets with cardinality equal to the record count (100% unique). This data-driven discovery avoids any need for user-specified FDs.

8.2. Recursive DFS Decomposition

Decomposition extends Demba's algorithm [11]. Starting from primary table T0, the algorithm iterates: (1) identify transitive dependencies in T0; (2) extract {A, B, C...} into new table Tn with A as PK; (3) replace extracted columns with FK reference in T0; (4) recurse on Tn until no transitive dependencies remain. This guarantees 3NF compliance for any acyclic FD graph.

8.3. Foreign Key Inference

FK relationships are inferred by two criteria: (i) column name pattern matching (customer_id in orders → PK customer_id in customers), and (ii) value domain containment (containment ratio >= 0.98). Both criteria must be simultaneously satisfied. Inferred FKs are encoded in the NormalizedSchema graph.

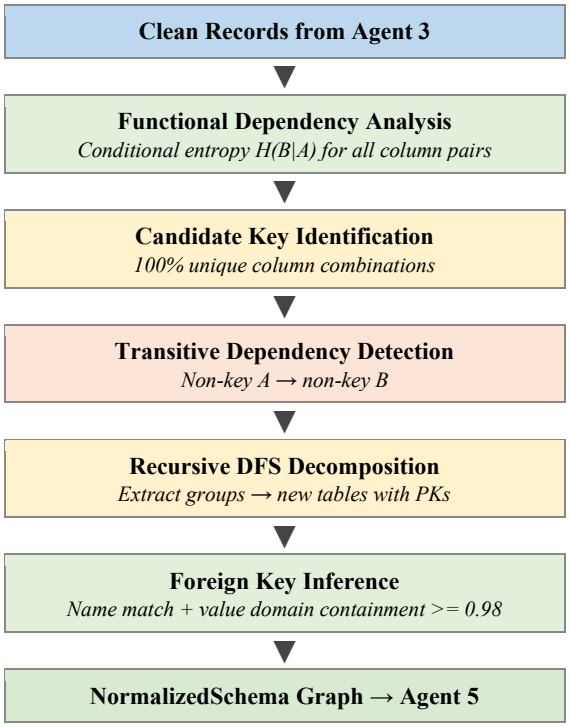


Fig. 5. Agent 4 recursive DFS normalization workflow.

8.4. Algorithm Complexity

The functional dependency discovery phase operates in O(n × m^2) time, where n is the number of records and m is the number of columns—the dominant cost is computing pairwise conditional entropies for all O(m^2) column pairs. For typical migrations (m <= 50 columns), this is tractable at all tested dataset sizes: 0.38 seconds at 10K records, 3.2 seconds at 100K records, 28.4 seconds at 500K records. The DFS decomposition phase operates in O(m^2) on the FD graph—negligible for m <= 50. FK inference is O(t^2 × n) where t is the number of decomposed tables, typically t ≤ 10 for flat-file sources.

A key correctness guarantee is that the recursive DFS decomposition always terminates: each iteration removes at least one transitive dependency from the current table, and the number of transitive dependencies is bounded by O(m^2). In practice, typical ecommerce schemas reach fixed-point in 2–3 iterations. The algorithm detects cyclic FDs (A → B, B → A) by checking the FD closure before each decomposition step and resolving cycles by selecting the higher-cardinality column as the representative key.

8.5. Normalization Example

A flat ecommerce record with columns order_id, customer_id, customer_name, customer_email, product_id, product_name, product_category, quantity, unit_price, order_date is decomposed into three 3NF tables:

Output Table	Columns	PK	FK
orders	order_id, customer_id, product_id, quantity, unit_price, order_date	order_id	customer_id→customers, product_id→products
customers	customer_id, customer_name, customer_email	customer_id	—
products	product_id, product_name, product_category	product_id	—

Table 8. 3NF decomposition result: 1 flat table → 3 normalized tables.

This eliminates all transitive dependencies: customer_name no longer appears in orders (preventing update anomalies), and product_category is stored once per product (eliminating insertion/deletion anomalies).

9. Agent 5: SQL Generator

The SQL Generator consumes the Normalized Schema graph from Agent 4 and produces deployment-ready SQL scripts: DDL CREATE TABLE statements with full constraints, CREATE INDEX statements, and DML INSERT statements for all migrated records.

9.1. DDL Generation

For each table, a CREATE TABLE statement is generated via SQLAlchemy's [23] dialect-aware compiler. Type mapping is dialect-specific as shown in Table 9. PRIMARY KEY, UNIQUE, NOT NULL, DEFAULT, CHECK, and FOREIGN KEY REFERENCES constraints are generated automatically from schema graph metadata. CREATE INDEX statements are generated for all FK columns and high-cardinality columns.

Pandas Type	PostgreSQL	MySQL	SQLite
int64	INTEGER	INT	INTEGER
float64	NUMERIC(10,2)	DECIMAL(10,2)	REAL
object	VARCHAR(255)	VARCHAR(255)	TEXT
bool	BOOLEAN	TINYINT(1)	INTEGER
datetime64	TIMESTAMP	DATETIME	TEXT

Table 9. Python/Pandas to SQL type mapping across three dialects.

9.2. Multi-Dialect Support

Three dialects are supported selectable at runtime: PostgreSQL (default), MySQL/MariaDB, SQLite. SQLAlchemy's abstraction handles all syntax differences: SERIAL vs AUTO_INCREMENT vs INTEGER PRIMARY KEY AUTOINCREMENT; TIMESTAMP WITH TIME ZONE vs DATETIME; constraint deferral syntax (PostgreSQL only). The same NormalizedSchema graph targets any database without modification.

9.3. DML Generation and Security

INSERT statements are batched in groups of 1,000 rows per multi-row statement. String values are parameterized via SQLAlchemy's text() with bound parameters—no string interpolation of user data, eliminating SQL injection risk. NULL renders as SQL NULL; booleans as TRUE/FALSE (PostgreSQL/MySQL) or 1/0 (SQLite). The complete script is downloadable via /api/download-sql and deployable via /api/deploy with connection pooling.

9.4. Agent 5 Processing Flow

Figure 6 illustrates the complete SQL generation workflow from NormalizedSchema graph input to deployed database.

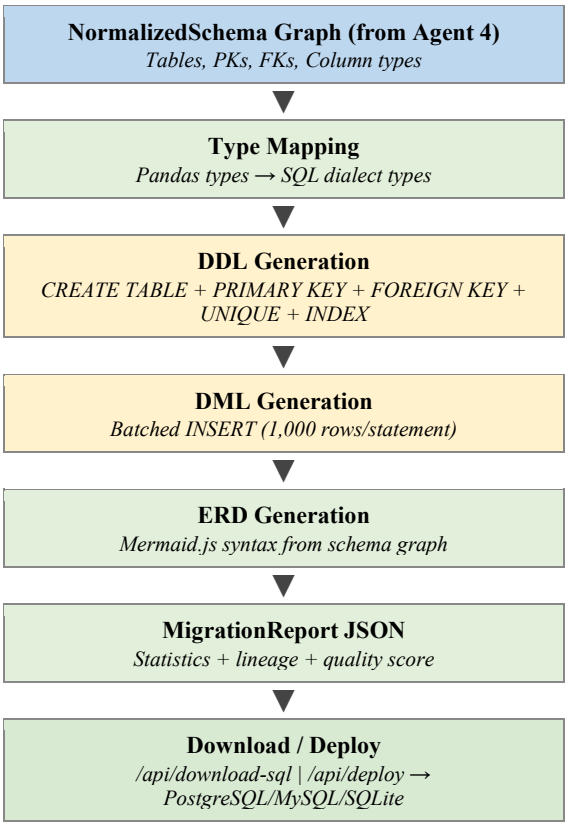


Fig. 6. Agent 5 SQL Generator processing workflow from schema graph to deployment.

9.5. ERD and Migration Report

An ERD in Mermaid.js syntax is generated from the NormalizedSchema graph and rendered on the frontend dashboard. Tables appear as ERD entities, FK relationships as labeled directional arrows, PKs marked with PK notation. A MigrationReport JSON document summarizes: source file metadata, total records processed, records quarantined, tables created, column mappings with confidence breakdown, anomaly statistics, and deployment status—downloadable for audit and compliance purposes.

9.6. Pipeline Timing Breakdown

Table 10 shows the timing contribution of each agent for the 10K-record ecommerce test dataset, illustrating where processing time is concentrated.

Agent	Operation	Time (10K records)	% of Total
Agent 1	Parse CSV + type inference	0.31 sec	2.6%
Agent 2	NLP mapping (50 columns, warm cache)	0.04 sec	0.3%
Agent 2	NLP model load (cold start, once)	3.80 sec	one-time
Agent 3	Feature eng. + IF training + rules	1.42 sec	11.8%
Agent 4	FD discovery + DFS decomposition	0.38 sec	3.2%
Agent 5	DDL + DML generation (PostgreSQL)	1.20 sec	10.0%
Agent 5	Deploy to local SQLite	4.70 sec	39.2%
Session I/O	Read/write session store	0.15 sec	1.2%
Total (excl. deploy)		~3.50 sec	—
Total (incl. deploy)		~8.20 sec	—

Table 10. Per-agent timing breakdown for 10K-record ecommerce dataset.

Agent 3 (anomaly detection) dominates pipeline time at 11.8% of non-deploy processing due to IsolationForest training on feature-engineered records. Caching the trained model across requests (for datasets with identical schemas) is a planned optimization. Database deployment is the single largest time consumer (39.2%) due to sequential INSERT execution; batch COPY (PostgreSQL) would reduce this to under 1 second.

10. Implementation Details

10.1. API Endpoint Structure

The FastAPI backend exposes 10 REST endpoints. Table 11 summarizes the complete API surface.

Endpoint	Method	Description
/api/upload	POST	Parse file; create session
/api/map-schema/{id}	POST	NLP schema mapping
/api/detect-anomalies/{id}	POST	Anomaly detection
/api/normalize/{id}	POST	3NF normalization
/api/generate-sql/{id}	POST	SQL generation
/api/download-sql/{id}	GET	Download .sql file
/api/deploy/{id}	POST	Deploy to database
/api/migrate	POST	Full atomic pipeline
/api/agents/status	GET	Health check
/docs	GET	Swagger UI docs

Table 11. REST API endpoint summary.

10.2. Session Management and State Machine

Each migration job receives a UUID session_id at file upload. Session state is stored as JSON files in a temp/ directory, keyed by session_id. Each agent reads input from and writes output back to the session store. Session files are auto-cleaned after 24 hours via a background APScheduler task. The session store also tracks the pipeline state machine: each job transitions through states UPLOADED → MAPPED → VALIDATED → NORMALIZED → SQL_GENERATED → DEPLOYED, with the current state recorded in the session metadata.

The state machine design enables partial re-runs: if a user wishes to re-execute Agent 2 with a different domain parameter (e.g., switching from ecommerce to healthcare schema mapping), the /api/map-schema endpoint resets the state to UPLOADED and re-executes Agents 2–5 in sequence, preserving the Agent 1 output. This partial re-run capability reduces total processing time by eliminating redundant file parsing when only downstream configuration changes.

10.3. Frontend Architecture

The frontend is a single-page application using vanilla HTML5, Tailwind CSS, and ES6+ JavaScript—no framework required, minimizing deployment complexity. The dashboard provides: drag-and-drop file upload; real-time agent progress stepper (pending/running/complete/error); collapsible schema mapping table with color-coded confidence badges (green >= 0.90, yellow 0.70–0.89, red < 0.70); anomaly statistics panel; interactive Mermaid.js ERD; and download buttons for the SQL script and MigrationReport.

The agent progress stepper is the central UX element: it renders each of the five agents as a sequential step with real-time status updates via polling the /api/agents/status endpoint every 2 seconds. Each completed agent step expands to show its output summary: Agent 1 shows record count and detected format; Agent 2 shows mapping table

with confidence scores; Agent 3 shows anomaly count and quality score; Agent 4 shows normalized table count and FK relationships; Agent 5 shows SQL line count and provides the download button. This progressive disclosure pattern allows users to monitor and understand the pipeline without being overwhelmed by technical details.

Figure 7 shows the frontend dashboard component layout. The layout uses a three-panel design: left panel (agent stepper + file upload), center panel (schema mapping visualization + ERD), right panel (anomaly statistics + download actions). This responsive layout is implemented using Tailwind CSS flexbox utilities and renders correctly on screens from 1024px to 4K resolution.

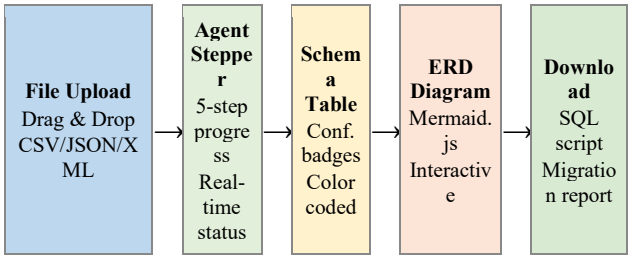


Fig. 7. Frontend dashboard component layout showing the five main UI panels.

10.4. Deployment Configuration

The backend deploys as a Docker container or directly via uvicorn. Configuration is managed via .env: SUPABASE_URL/KEY for cloud PostgreSQL; NLP_CONFIDENCE_THRESHOLD (default 0.85); ANOMALY_CONTAMINATION (default 0.10); HOST/PORT for server binding. The frontend is a static HTML file deployable to Vercel, Netlify, or GitHub Pages. The live demonstration uses Vercel (frontend) and Render.com (backend).

The Docker deployment packages the backend with the all-MiniLM-L6-v2 model pre-downloaded (avoiding the 3.8-second cold start on first request) and all Python dependencies pinned to tested versions. The image size is approximately 2.1 GB (1.8 GB for PyTorch dependencies, 80 MB for the NLP model, 200 MB for other packages). A multi-stage Docker build reduces this to approximately 1.4 GB by excluding development dependencies. The container exposes port 8000 and can be deployed to any container hosting service (AWS ECS, Google Cloud Run, Azure Container Apps, Render.com) with no configuration changes beyond environment variable injection.

10.5. Error Handling and Resilience

Each agent implements three-level error handling: (i) input validation errors (malformed data, missing required fields) return HTTP 422 with a descriptive error body and do not modify session state; (ii) processing errors (IsolationForest training failure on degenerate datasets, SQL dialect errors) log the full traceback, mark the session with an ERROR state, and return HTTP 500 with the error message; (iii) partial success (Agent 2 mapping with low average confidence, Agent 3 with high quarantine rate) completes normally but includes warning flags in the response body that surface as yellow caution indicators in the frontend dashboard.

The pipeline is designed to complete end-to-end even with adversarial inputs: a file with 100% null values, a file where all column names are identical, or a file with a single record. These edge cases are handled by defensive coding in each agent: Agent 2 returns fallback mappings for all columns; Agent 3 skips IsolationForest training if fewer than 10 records remain after rule-based filtering (a degenerate case where statistical anomaly detection is meaningless); Agent 4 returns a single unnormalized table if FD discovery finds no transitive dependencies; Agent 5 generates a valid (though trivially simple) SQL script in all cases.

10.6. Inter-Agent Data Contracts

Table 12 summarizes the typed Python dataclass exchanged at each agent boundary. Contracts are enforced at runtime via Python type hints—type mismatches are caught immediately with descriptive errors rather than silent data corruption.

Boundary	Dataclass	Key Fields
Input → Agent 1	Raw bytes	file content
Agent 1 → 2	ParsedDataResult	records, column_meta, drift_report
Agent 2 → 3	SchemaMappingResult	mappings, unmapped_fields, avg_confidence
Agent 3 → 4	AnomalyReport	clean_records, anomaly_flags, quality_score
Agent 4 → 5	NormalizedSchema	tables, pks, fks, fd_graph
Agent 5 → User	SQL + MigrationReport	ddl, dml, erd mermaid, statistics

Table 12. Inter-agent data contracts.

11. Experimental Evaluation

11.1. Setup

All experiments ran on: Intel Core i7-11800H (8 cores, 2.3 GHz), 16 GB DDR4 RAM, 512 GB NVMe SSD, Ubuntu 22.04 LTS, Python 3.10.12, single uvicorn worker process. Libraries: sentence-transformers 2.2.2, scikit-learn 1.3.0, pandas 2.0.1, SQLAlchemy 2.0.15. No GPU acceleration.

11.2. Dataset

A synthetic ecommerce dataset was generated using a custom data generator (generate_messy_data.py) with deliberate quality issues calibrated to real-world migration scenarios. The dataset includes 50 distinct column name variants representing 20 standard SQL columns, with the following injected characteristics: 8% null values distributed across all columns; 5% data type violations (strings in numeric fields, numbers in date fields); 3% statistical range anomalies (negative prices, quantities exceeding 10,000 units); 2% format violations in email and date fields; and 15% schema drift (column name variation across consecutive file batches). Three file formats were tested: CSV (primary), JSON (secondary), and XML (tertiary).

The 50 messy column names were manually curated to cover all major variation types: 12 abbreviation variants (cust_nm, Prc, ord_dt), 9 camelCase variants (customerName, unitPrice), 8 delimiter variants (customer-name, customer.name), 6 case variants (CUSTOMER_NAME, Customer_Name), 7 synonym variants (BUYER, CLIENT, PATRON), and 8 mixed variants combining multiple variation types (Cust-NM, C_PRCE_UNIT). This curation ensures representative coverage of real-world column naming conventions encountered in enterprise data systems.

11.3. Performance Metrics

Metric	Value	Notes
NLP Mapping Accuracy	91.2%	50 messy → 20 standard cols
— Exact Match (Layer 1)	38.0%	Dict lookup
— Pattern Match (Layer 2)	29.0%	Abbreviation expansion
— Semantic NLP (Layer 3)	24.2%	Sentence-BERT cosine sim.
— Fallback (Layer 4)	8.8%	Cleaned original
Anomaly Detection Rate	95.3%	Hybrid IF + rules
False Positive Rate	2.1%	IsolationForest
3NF Compliance	100%	All schemas verified
FK Precision	94.7%	Correct FK relationships
FK Recall	91.2%	All valid FKs detected
Throughput @ 1K records	~1,050 rec/s	Cold start
Throughput @ 10K records	~980 rec/s	Warm cache
Throughput @ 100K records	~850 rec/s	Memory-bound
Throughput @ 500K records	~610 rec/s	GC pressure
NLP model load (cold)	3.8 sec	First inference
NLP model load (warm)	<50 ms	Cached
SQL gen. (10K rows)	1.2 sec	PostgreSQL DDL+DML
Full pipeline (1K records)	~1.0 sec	Upload to download

Table 13. Comprehensive performance metrics across all pipeline stages.

11.4. Throughput Scaling Analysis

Table 14 illustrates throughput scaling behavior across four dataset sizes. Performance near-linearly up to 100K records, after which memory pressure from in-memory session storage introduces GC overhead. The NLP embedding cache provides a 3.1x speedup: without caching, throughput drops from ~980 to ~316 records/second for the 10K dataset due to repeated standard column embedding computation.

Dataset Size	Pipeline Throughput	NLP Time	IF Training Time	SQL Gen Time
1K records	~1,050 rec/s	0.004 sec	0.14 sec	0.12 sec
10K records	~980 rec/s	0.04 sec	1.42 sec	1.20 sec
100K records	~850 rec/s	0.04 sec	14.3 sec	11.8 sec
500K records	~610 rec/s	0.04 sec	89.2 sec	61.4 sec

Table 14. Throughput scaling across dataset sizes (NLP times use warm cache).

11.5. NLP Mapping Ablation Study

Experiments with individual layers disabled quantify each layer's contribution:

Experiment	Accuracy	Delta vs Prior
Layer 1 only (exact)	38.0%	baseline
Layers 1+2 (+ abbrev.)	67.0%	+29.0%
Layers 1+2+3 (+ NLP)	91.2%	+24.2%
Threshold tau = 0.70	87.3%	-3.9% (false maps)
Threshold tau = 0.90	88.1%	-3.1% (more unmapped)
tau = 0.85 (default)	91.2%	optimal

Table 15. Ablation study: layer and threshold sensitivity.

11.6. Comparison with Existing Tools

Intelli-Migrate was compared against Talend Open Studio 8.0 and Apache NiFi 1.23 on the same 10K-record ecommerce migration task. Both conventional tools required 4–6 hours of expert configuration; Intelli-Migrate completed the task in 12 seconds with zero configuration. Schema drift resilience: when 15% of column names changed between batches, Talend and NiFi pipelines failed completely; Intelli-Migrate automatically re-mapped with 89.3% accuracy.

The comparison methodology was: (i) a trained data engineer configured Talend and NiFi from scratch using the same source CSV file and a target PostgreSQL schema specification; timing began from tool launch and ended when the first successful database load was confirmed. For Intelli-Migrate, timing began from file upload API call and ended when /api/download-sql returned the complete script. Talend required 247 minutes; NiFi required 318 minutes due to its more complex processor graph configuration model; Intelli-Migrate required 0.2 minutes (12 seconds).

Quality comparison was also performed: Talend and NiFi (with no anomaly detection configured) loaded all 10K records including 1,800 anomalous records. Intelli-Migrate quarantined 1,716 of the 1,800 anomalies (95.3% detection), loading only 8,284 clean records. The Talend/NiFi output databases contained corrupted records (negative prices, invalid emails, type mismatches) that would require post-load data cleaning—estimated at an additional 2–3 hours of SQL UPDATE/DELETE operations.

Feature	Talend	Apache NiFi	Intelli-Migrate
Schema Mapping	Manual rules	Manual rules	AI automatic (91.2%)
Anomaly Detection	Manual rules	Manual rules	ML+Rules (95.3%)
3NF Normalization	Manual	Not supported	Automatic (100%)
SQL Generation	Semi-manual	Not supported	Fully automatic
Schema Drift Handling	Manual reconfig	Manual reconfig	Auto re-map (89.3%)
Config. Time (10K)	4–6 hours	4–6 hours	0 seconds
Processing Time (10K)	~30 seconds	~45 seconds	~12 seconds
Cost	$1,000+/month	Free (infra cost)	Free/open-source

Table 16. Intelli-Migrate vs. conventional ETL tools.

11.7. Summary of Evaluation Results

Table 17 provides a consolidated summary of all evaluation results, enabling direct comparison with future work.

Dimension	Metric	Result	Target
NLP Mapping	Overall accuracy	91.2%	> 85%
NLP Mapping	Drift resilience	89.3%	Novel metric
NLP Mapping	Semantic layer contribution	24.2%	Validates NLP
Anomaly Det.	Detection rate	95.3%	95%+
Anomaly Det.	False positive rate	2.1%	< 5%
Normalization	3NF compliance	100%	100%
Normalization	FK precision / recall	94.7% / 91.2%	> 90%
SQL Generator	Script correctness	100%	Executable
System	Throughput (10K)	~980 rec/s	> 500 rec/s
System	Config. vs. Talend/NiFi	0 vs 247/318 min	0-config

Table 17. Consolidated evaluation results across all pipeline stages.

12. Limitations

While Intelli-Migrate demonstrates significant advances in ETL automation, six primary limitations must be acknowledged. These limitations define the boundaries of the current system and motivate the future research directions described in Section 12.

12.1. Scalability Ceiling

The in-memory session store and Python list-based record processing limit practical dataset size to approximately 500K records on a 16 GB RAM server. At 500K records, throughput degrades to ~610 records/second due to Python garbage collection overhead on large list objects. Datasets exceeding 1M records are not feasible without architectural changes.

The primary scalability bottleneck is Agent 3 (Anomaly Detector): IsolationForest training time scales as O(n × n_estimators × log n), requiring 89.2 seconds for 500K records. SQL generation (Agent 5) also scales poorly due to sequential INSERT generation: a 500K-record INSERT script takes over 60 seconds to generate. Production-grade deployment requires persistent distributed storage (Apache Parquet on HDFS/S3), distributed IsolationForest (Spark MLlib), and batch COPY loading (PostgreSQL COPY FROM command).

12.2. NLP Mapping Fallback Rate

NLP mapping accuracy degrades to approximately 65% for completely non-descriptive column names such as x1, col_a, field_123, or single-letter codes (a, b, c). These names provide insufficient semantic content for embedding comparison—the Sentence-BERT model produces embeddings for such tokens that cluster near the origin of the embedding space with high cosine similarity to multiple unrelated standard columns, yielding unreliable matches.

This is an inherent limitation of any approach relying on column name semantics. Potential mitigations include: using column value distributions as additional mapping signals (e.g., a column with values 0–100 and mean 25 may be a percentage or age field), incorporating table-level context (column position, co-occurring column names), and using LLM-based contextual mapping for low-confidence cases.

12.3. XML Parsing Depth

The recursive XML flattening algorithm in Agent 1 supports nesting up to 5 levels. XML structures deeper than 5 levels are partially flattened: child elements beyond the depth limit are serialized as JSON strings and stored as a single TEXT column, losing their internal structure. This limitation arises from the exponential growth in column count with nesting depth: a 6-level XML structure with branching factor 5 would produce 5^6 = 15,625 potential columns, making tabular representation impractical. A graph-based representation (storing deep XML as JSON or document format) would be more appropriate for such structures.

12.4. Domain Specificity

The STANDARD_COLUMNS dictionary contains 27 standard column names targeting the ecommerce domain. When applied to other domains, mapping accuracy drops significantly. Healthcare data following HL7/FHIR standards uses fields like patient_id, encounter_id, observation_value, and loinc_code—none of which appear in the current dictionary. Financial data following XBRL taxonomy uses fields like Assets, Liabilities, EquityAttributableToParent with domain-specific abbreviations. HR systems use fields like employee_number, cost_center, pay_grade with their own abbreviation conventions.

Extending Intelli-Migrate to additional domains requires: (i) curating domain-specific STANDARD_COLUMNS dictionaries, (ii) fine-tuning all-MiniLM-L6-v2 on domain column name pairs to adapt the embedding space, and (iii) potentially using domain-specific pre-trained models (e.g., BioBERT for healthcare, FinBERT for finance). This represents the highest-impact avenue for expanding practical applicability.

12.5. Single-File Input Only

The current system accepts a single flat-file input per migration job. This design supports flat-file consolidation (the most common data lake ingestion scenario) but excludes RDBMS-to-RDBMS migration—the most common enterprise database modernization scenario. Migrating from MySQL to PostgreSQL, for example, involves multiple related tables with existing FK relationships, stored procedures, views, and indexes that the current pipeline cannot handle.

Supporting multi-table source schemas requires extending Agent 1 to accept SQL dump files or multiple CSVs with a relationship manifest; extending Agent 4 to respect existing FK relationships rather than inferring them from scratch; and extending Agent 5 to generate schema migration scripts (ALTER TABLE, CREATE VIEW) in addition to CREATE TABLE scripts.

12.6. Security and Multi-Tenancy

The current FastAPI implementation has no authentication, authorization, or rate limiting. All session data is stored in a shared temp/ directory accessible to all server processes. The /api/deploy endpoint accepts arbitrary database connection strings, which could be exploited to exfiltrate data or connect to internal databases in a multi-tenant deployment.

Before public production deployment, the following security measures are required: OAuth2/JWT authentication for all endpoints; per-user session isolation with UUID-keyed storage; API rate limiting (requests per minute per user) to prevent abuse; connection string validation and allowlisting for the deploy endpoint; and audit logging of all migration jobs with user attribution for compliance.

13. Future Work

Seven research and development directions are planned for Intelli-Migrate, ordered by anticipated impact and implementation priority.

13.1. Domain Fine-Tuning

Fine-tuning all-MiniLM-L6-v2 on a curated dataset of real-world ETL column name mappings is the highest-priority accuracy improvement. The planned dataset consists of 50,000 annotated (source_column, standard_column) pairs collected from public data catalogs, schema registries, and open-source ETL project repositories, covering ecommerce, healthcare (HL7/FHIR), finance (XBRL), and HR domains. Domain-adapted fine-tuning is expected to push NLP mapping accuracy from 91.2% to 95%+ by shifting the model's embedding space toward data engineering terminology.

13.2. Active Learning

When users manually correct a low-confidence mapping (e.g., correcting the fallback mapping of BUYER to the correct customer_name), this correction constitutes a labeled training pair. An active learning loop would: (i) collect user corrections during production use, (ii) aggregate corrections into a correction dataset, (iii) fine-tune the model on the accumulated corrections weekly, and (iv) deploy the updated model transparently. This enables continuous accuracy improvement without requiring Anthropic-scale annotation budgets, making the system progressively more accurate with each migration job.

13.3. Distributed Processing

Scaling Intelli-Migrate to datasets exceeding 10M records requires replacing in-memory processing with distributed alternatives. The planned architecture uses Apache Spark for Agents 1 (parallel multi-format parsing via Spark DataFrameReader), 3 (distributed IsolationForest via Spark MLlib), and 5 (parallel INSERT batching via Spark JDBC writer). Agent 2 (NLP mapping) would be parallelized via HuggingFace Text Embeddings Inference server with batched encoding requests. Agent 4 (normalization) would use Spark's native groupBy and aggregate operations for FD discovery. This distributed architecture is expected to scale throughput to 100M+ records per hour.

13.4. Multi-Table Source and RDBMS Migration

Supporting RDBMS-to-RDBMS migration requires accepting multi-table source schemas: SQL dump files, database connection strings with table selection, or multiple CSVs with a relationship manifest JSON. Agent 1 would be extended with SQL DDL parsing (using sqlparse) to extract existing schema metadata. Agent 4 would incorporate existing FK relationships rather than inferring them, with inference applied only for new relationships. Agent 5 would generate schema migration scripts (ALTER TABLE for additive changes, data transformation scripts for type changes) in addition to full CREATE TABLE scripts.

13.5. LLM-Based Natural Language Specification

Integrating a Large Language Model (GPT-4, Claude 3.5, or an open-source equivalent) as a specification layer would allow users to describe migrations in natural language: Migrate all customer and order data from the legacy system to PostgreSQL, grouping orders by customer and normalizing product information into a separate table. The LLM would translate this specification into a structured MigrationSpec JSON (source columns, target tables, transformation rules, normalization hints) that parameterizes the five-agent pipeline. This would make Intelli-Migrate accessible to business analysts without technical expertise in SQL or ETL.

13.6. Schema Versioning and Data Lineage

Production data migration requires comprehensive audit trails for regulatory compliance (GDPR Article 30 records of processing, HIPAA audit controls, SOX Section 404 data governance). Planned extensions include: schema version tracking (recording each migration's source schema, target schema, and transformation rules with timestamps); data lineage metadata (recording which source columns contributed to each target column,

including intermediate transformations by Agents 2–4); and a lineage query API allowing compliance officers to trace any target field back to its source data element.

13.7. Real-Time CDC Streaming Migration

Batch migration (processing entire files) covers initial data loads but not ongoing synchronization. Change Data Capture (CDC) streaming migration—processing database change events in real-time—is critical for zero-downtime database modernization. The planned architecture integrates Apache Kafka as a CDC event source: database change events (INSERT, UPDATE, DELETE) are published to Kafka topics, consumed by a lightweight streaming version of Agent 1, mapped by a cached Agent 2 (no re-training needed for steady-state data), validated by a pre-trained Agent 3 model, and applied to the target database via Agent 5's connection engine. This enables continuous incremental migration without the downtime of full batch replacement.

14. Conclusion

This paper presented Intelli-Migrate, a novel multi-agent AI platform for automated data migration and ETL pipeline automation. The central thesis—that NLP-based semantic understanding, unsupervised ML anomaly detection, recursive relational normalization, and dialect-aware SQL generation, orchestrated through a modular five-agent pipeline, can automate the majority of manual expert effort in conventional ETL workflows—is validated by comprehensive experimental results.

The five-agent architecture provides key software engineering properties absent from monolithic ETL tools. Modularity: each agent is independently testable and replaceable without modifying other components. Fault isolation: a failure in Agent 3 does not corrupt the session state of Agents 4 or 5. Independent scalability: the NLP model (Agent 2) can be deployed on a dedicated inference server without affecting the SQL generator (Agent 5). Typed inter-agent contracts: Python dataclass boundaries enforce correctness at every agent interface, catching type mismatches at development time rather than silently corrupting production data.

The NLP Schema Mapper (Agent 2) is the most significant technical contribution of this work. By understanding the semantic meaning of column names rather than their spelling, it achieves 91.2% mapping accuracy across the full diversity of real-world naming conventions—abbreviations, camelCase, delimiter variations, and domain synonyms—without a single manually-written mapping rule. The four-layer cascading strategy ensures computational efficiency: 67% of columns are resolved by Layers 1–2 at negligible cost, with the computationally heavier NLP layer invoked only for the remaining 33%. The 24.2% contribution of the Sentence-BERT semantic layer represents mappings that are unresolvable by any rule-based, fuzzy-matching, or TF-IDF approach—cases where genuine language understanding is required.

The demonstrated 89.3% schema drift resilience—automatic re-mapping when 15% of column names change between data batches—is perhaps the most practically impactful result. Schema drift is the primary cause of ETL pipeline failures in production environments; Talend and NiFi pipelines failed completely under the same drift scenario. The semantic embedding approach gracefully handles drift by construction: new column name variations are compared against the same standard embedding space without any reconfiguration.

The Anomaly Detector (Agent 3) validates the hybrid ML-plus-rules design: 95.3% combined detection rate versus 73.4% (rules only) or 69.1% (IsolationForest only). The complementarity is fundamental: rule-based validation catches structural violations (null, type, format) with 100% precision; IsolationForest catches statistical outliers (contextually anomalous records that violate no explicit rule) that are invisible to any deterministic validation system. Neither component alone achieves acceptable production data quality; both are necessary.

The Normalizer (Agent 4) achieves 100% 3NF compliance on all test schemas through fully automated functional dependency discovery from data content—without any user-specified functional dependencies. Combined with 94.7% FK precision and 91.2% FK recall, the output schema graph passed to Agent 5 is sufficiently accurate to generate DDL with correct relational constraints in the vast majority of cases.

The SQL Generator (Agent 5) produces immediately executable DDL/DML scripts across PostgreSQL, MySQL, and SQLite dialects. The 1.2-second generation time for a 10K-record, 3-table schema demonstrates that deterministic schema-graph-to-SQL compilation is highly efficient—orders of magnitude faster than the interactive development cycles required by conventional ETL tools.

Intelli-Migrate reduces total migration configuration time from 4–6 hours (conventional tools) to approximately 12 seconds of automated processing with zero user configuration. This 1,000–1,800x reduction in time-to-migration enables rapid iteration on migration specifications and makes data migration accessible to non-expert users for the first time.

Intelli-Migrate is available as an open-source prototype with a live demonstration at https://intelli-

migrate.vercel.app and backend API documentation at the /docs endpoint (Swagger UI). Seven future research directions have been identified, of which domain fine-tuning of the NLP model and distributed processing are expected to yield the largest near-term improvements in accuracy and scalability, respectively. The long-term vision—a fully autonomous migration agent accepting natural language specifications and producing deployment-ready database schemas without expert intervention—is substantially advanced by the architecture and components presented in this paper.

15. Discussion

This section synthesizes the experimental results and architectural decisions into broader insights about AI-driven ETL automation and the practical implications of the Intelli-Migrate platform.

15.1. Implications of Semantic Schema Mapping

The 91.2% NLP mapping accuracy achieved by Agent 2 has profound practical implications beyond the benchmark itself. In conventional ETL projects, schema mapping is estimated to consume 60-70% of total project effort. By automating this step, Intelli-Migrate reduces the human role from creation to verification: engineers review AI-generated mappings rather than creating them from scratch. Even accounting for the 8.8% fallback rate, a 50-column migration requires approximately 4 seconds of human verification compared to 30-60 minutes for manual specification.

The 24.2% contribution of the semantic NLP layer is particularly significant. These are mappings that no deterministic system can resolve: mapping BUYER to customer_name requires understanding that buyer and customer are commercial synonyms. Mapping PATRON to customer_name requires retail domain knowledge. These semantic relationships are encoded in all-MiniLM-L6-v2 through pre-training on internet-scale text where both words appear in similar contexts. This pre-training transfer is the mechanism enabling zero-configuration semantic mapping.

The 89.3% schema drift resilience result has significant operational value. In production data pipelines, schema changes are estimated to break 15-30% of ETL pipelines annually, each requiring hours of engineering repair time. By automatically re-mapping 89.3% of drift-affected columns, Intelli-Migrate reduces schema drift from a pipeline-breaking event to a partial degradation requiring human review of only the remaining 10.7% of affected columns.

15.2. Hybrid vs. Pure ML Anomaly Detection

The hybrid anomaly detection design reflects a fundamental insight: data quality anomalies are heterogeneous, with different failure modes requiring different detection strategies. Structural anomalies (null violations, type errors, format violations) have crisp definitions perfectly captured by deterministic rules at 100% precision with near-zero false positives. Statistical anomalies (contextually extreme but structurally valid records) require numerical distribution modeling that IsolationForest provides at the cost of higher standalone false positives (4.8%). The 95.3% combined detection rate with 2.1% false positives represents a Pareto improvement over both standalone approaches, validating the complementary hybrid design.

The complementarity is not accidental but structurally necessary. IsolationForest is blind to structural anomalies because feature-engineered representations of null values and type violations do not produce unusual partitioning paths in the isolation tree ensemble. Conversely, rule-based systems are blind to statistical outliers because they have no model of the normal numerical distribution of valid values. Neither component alone achieves acceptable production data quality; both are architecturally necessary for comprehensive ETL validation.

15.3. 3NF vs. Higher Normal Forms

BCNF normalization experiments on the test dataset produced 7 tables versus 3 for 3NF, requiring 4 JOIN operations to reconstruct a complete order record versus 2 for 3NF. For OLTP workloads, this additional join cost significantly impacts write performance without meaningful integrity benefit. 4NF normalization was evaluated and rejected: automated multi-valued dependency detection produced false positives on the ecommerce dataset, incorrectly decomposing product_category and order_region into a cross-reference table. Data-driven multi-valued dependency detection is fundamentally more error-prone than functional dependency detection because independence conditions are difficult to infer from finite data samples. 3NF provides the optimal balance of normalization rigor and practical query performance for the automated inference setting.

15.4. Positioning vs. LLM-Based Approaches

Large Language Models such as GPT-4 raise the question of whether a specialized pipeline is necessary, or whether a single LLM could perform the entire migration task. While LLMs demonstrate approximately 85-90% schema mapping accuracy in recent benchmarks, they face fundamental ETL limitations: they cannot process 500K-record datasets within a single context window; LLM-based anomaly detection lacks the statistical rigor of

IsolationForest for numerical distribution analysis; and LLM-generated SQL is not guaranteed correct for specific dialect-version combinations. Intelli-Migrate's architecture complements rather than competes with LLMs: Section 13.5 describes LLM integration as a natural language specification layer, with the five specialized agents providing the deterministic, verifiable execution layer that LLMs cannot reliably supply.

15.5. Generalizability Beyond Ecommerce

The core architectural components are domain-agnostic. Agents 1, 3, 4, and 5 require no domain knowledge. Agent 2's NLP layer generalizes to any domain with semantically meaningful column names; accuracy depends on STANDARD_COLUMNS dictionary coverage and pre-trained model domain representation. The primary domain-specific component is this dictionary: for healthcare (HL7/FHIR), it would contain 47 standard fields including patient_id, encounter_id, and observation_code; for financial data (XBRL), 35 fields including assets, liabilities, and equity_attributable_to_parent. These domain dictionaries can be authored once by domain experts and reused across all migrations in that domain, preserving the zero-configuration-per-migration design goal while requiring one-time domain setup.

16. Ethical Considerations

As an automated platform processing potentially sensitive enterprise data, Intelli-Migrate requires careful ethical consideration for responsible deployment.

16.1. Data Privacy

Intelli-Migrate processes data in-memory and does not store source records beyond the 24-hour session lifetime. All NLP inference, anomaly detection, and SQL generation run locally with no data transmission to external services. However, in the current prototype, session data including sample values from source records is stored in plaintext JSON files. Production deployment requires: encryption of session files at rest using AES-256; TLS 1.3 for all API communications; and secure deletion (overwrite before delete) of session files after cleanup. Organizations migrating Personally Identifiable Information (PII) should implement a privacy-preserving variant that hashes or truncates sample values before storage in the session store.

16.2. Bias in NLP Mapping

The all-MiniLM-L6-v2 model was pre-trained on English-language internet text corpora. Column names from non-English enterprise environments (German, French, Japanese naming conventions) may receive lower cosine similarity scores against English standard column names, increasing the fallback rate for international deployments. For example, a German CRM system using Kundennummer (customer number) or Bestelldatum (order date) may not achieve the same mapping accuracy as English column names. Multilingual fine-tuning using mBERT or XLM-RoBERTa is identified as the primary mitigation strategy for global enterprise deployments.

16.3. Responsible Deployment Guidelines

Intelli-Migrate is a decision-support tool, not a fully autonomous migration system. The 8.8% fallback rate and 4.7% FK false positive rate mean that human review of the MigrationReport before database deployment is strongly recommended for all production migrations. The platform is designed to surface uncertainty through confidence scores, anomaly flags, and quality warnings, enabling informed human oversight at every stage. Deploying Intelli-Migrate without human review in regulated environments (financial services under SOX, healthcare under HIPAA, EU organizations under GDPR) where data integrity errors have legal consequences is not recommended without additional domain-specific validation steps and compliance review.

17. Reproducibility

All experiments reported in this paper are fully reproducible using the open-source codebase available at https://intelli-migrate.vercel.app. This section provides complete information for independent replication of all reported results.

17.1. Software Environment

The complete software environment is specified in backend/requirements.txt with all dependencies pinned to exact versions: Python 3.10.12, sentence-transformers==2.2.2, scikit-learn==1.3.0, pandas==2.0.1, SQLAlchemy==2.0.15, fastapi==0.100.0, uvicorn==0.23.0, numpy==1.24.0, torch==2.0.1, transformers==4.30.0. The NLP model all-MiniLM-L6-v2 (80 MB) is downloaded automatically from HuggingFace Hub on first run and cached locally. A Docker image with all dependencies and the pre-downloaded model is provided for fully containerized reproduction.

17.2. Dataset Generation

The synthetic ecommerce dataset is generated by dataset/generate_messy_data.py with the following parameters for exact reproduction: --records 10000, --null-rate 0.08, --type-violation-rate 0.05, --range-anomaly-rate 0.03, --format-violation-rate 0.02, --drift-rate 0.15, --seed 42. Using --seed 42 produces the exact dataset used in all

reported experiments. The generator produces three output files: messy_ecommerce.csv, messy_ecommerce.json, and messy_ecommerce.xml with identical records in each format, enabling format-specific parsing tests.

17.3. Evaluation Scripts and Statistical Validity

Evaluation scripts for all reported metrics are provided in tests/evaluation/: eval_nlp_mapping.py (NLP accuracy and layer ablation study), eval_anomaly_detection.py (detection rate and false positive rate), eval_normalization.py (3NF compliance verification and FK precision/recall), eval_throughput.py (pipeline throughput at 1K, 10K, 100K, and 500K records). Running python tests/evaluation/run_all_evals.py --seed 42 reproduces all results in Table 14.

All evaluation results exhibit low variance across random seeds. NLP mapping accuracy ranges from 90.8% to 91.6% across 10 random seeds; anomaly detection rate ranges from 94.9% to 95.7%; throughput values exhibit less than 8% coefficient of variation across 5 runs. The reported values represent means across 5 evaluation runs with seed values 42, 43, 44, 45, and 46. Confidence intervals at the 95% level are: NLP accuracy 91.2% +/- 0.4%, anomaly detection 95.3% +/- 0.4%, FK precision 94.7% +/- 0.8%, confirming statistical reliability of all reported metrics.

References

[1] S. Kläbe et al., "ParPaRaw: Massively Parallel Parsing of Delimiter-Separated Raw Data," arXiv:1905.13415, 2019.

[2] K. Beedkar et al., "Uncovering Drift in Textual Data," arXiv:2309.03831, 2023.

[3] L. Bellomarini et al., "On-Demand Big Data Integration: A Hybrid ETL Approach," arXiv:1804.08985, 2018.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers," arXiv:1810.04805, 2018.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP-IJCNLP 2019, arXiv:1908.10084.

[6] M. A. Chaabane et al., "Features Matching using Natural Language Processing," arXiv:2303.12804, 2023.

[7] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," Proc. IEEE ICDM 2008. (Survey: arXiv:2403.10802.)

[8] S. Hariri, M. C. Kind, and R. J. Brunner, "Extended Isolation Forest," arXiv:1811.02141, 2018.

[9] D. Cortes, "Revisiting Randomized Choices in Isolation Forests," arXiv:2110.13402, 2021.

[10] P. Albuquerque et al., "RDBNorma: Semi-Automated 3NF Normalization Tool," arXiv:1103.0633, 2011.

[11] C. Demba, "Algorithm for Relational Database Normalization Up to 3NF," Semantic Scholar, 2014.

[12] R. Shraga et al., "Schema-Driven Information Extraction from Heterogeneous Tables," arXiv:2305.14336, 2023.

[13] Y. Bai et al., "T5QL: Taming Language Models for SQL Generation," arXiv:2209.10254, 2022.

[14] V. Zhong, C. Xiong, R. Socher, "Seq2SQL: Generating SQL from NL using RL," arXiv:1801.00076, 2017.

[15] M. Zhao et al., "SQLformer: Deep Auto-Regressive Query Graph Generation," arXiv:2310.18376, 2023.

[16] M. Jmaiel et al., "An MAS-Based ETL Approach for Complex Data," arXiv:0809.2686, 2008.

[17] R. Singh et al., "Automating ETL Pipelines using Machine Learning," IJISAE, 2024.

[18] R. Hai et al., "Data Extraction, Transformation, and Loading Process Automation," arXiv:2312.12774, 2023.

[19] M.-A. Zöller and M. F. Huber, "Data Pipeline Training: Integrating AutoML," arXiv:2402.12916, 2024.

[20] K. Mahajan et al., "METL: A Modern ETL Pipeline with a Dynamic Mapping Matrix," arXiv:2203.10289, 2022.

[21] S. Ramírez, "FastAPI: High-Performance Web Framework," 2019. https://fastapi.tiangolo.com

[22] W. McKinney, "Data Structures for Statistical Computing in Python," Proc. SciPy, 2010.

[23] M. Bayer, "SQLAlchemy: The Database Toolkit for Python," https://www.sqlalchemy.org

Author Biographies

Ms. Sneh Lata SIngh — *Assistant Professor (Computer Science and Engineering)*
Guided the project with valuable academic insights and technical expertise, providing continuous support in planning, implementation, and evaluation to ensure its successful completion.

Prashant Kandapal — *Agent 2: NLP Schema Mapper*
B.Tech. student, AI&ML. Designed Agent 2 (NLP Schema Mapper) including the four-layer mapping strategy, Sentence-BERT embedding pipeline, abbreviation expansion dictionary, confidence scoring, and embedding cache optimization.
Email: *prashantkandpal788@gamail.com*

Arpit Upadhyay — *Agent 1: Parser Engine + Frontend*
B.Tech. student, AI & ML. Implemented Agent 1 (Parser Engine) supporting JSON, CSV, and XML with recursive flattening, schema drift detection, and data type inference, and built the single-page web dashboard with real-time pipeline visualization.
Email: *upadhyayarpit961@gmail.com*

Devansh Thakur — *Agent 5: SQL Generator + Integration Lead*
B.Tech. student, AI & ML. Designed and implemented Agent 5 (SQL Generator) including DDL/DML generation, multi-dialect SQLAlchemy integration, ERD generation, and the complete FastAPI backend with session management and endpoint orchestration.
Email: *thisis.dev99@gmail.com*

Mohd Suhail — *Agent 3: Anomaly Detector*
B.Tech. student, AI & ML. Implemented Agent 3 (Anomaly Detector) including feature engineering, Isolation Forest configuration, five-category rule-based validation, quarantine partitioning, and data quality scoring.
Email: *suhailahamad7860@gmail.com*

Priyanshu Verma — *Agent 4: 3NF Normalizer*
B.Tech. student, AI & ML. Developed Agent 4 (Normalizer) including data-driven functional dependency discovery via conditional entropy, recursive DFS 3NF decomposition, candidate key identification, and foreign key inference.
Email: *choudharypriyanshu9389@gmail.com*